



Arrays in C



What is an Array?

👉 An **array** is a collection of **same data type elements** stored in **contiguous memory locations**.

👉 Accessed using **index (0-based)**.

```
int arr[5];    // array of 5 integers
```



Memory Representation of Array

📌 Elements stored **continuously**

📌 Address calculation:

$\text{Address} = \text{BaseAddress} + (\text{index} \times \text{size of datatype})$

```
int arr[3] = {10, 20, 30};  
// if base = 1000, int = 4 bytes  
// arr[1] address = 1000 + (1*4) = 1004
```



Types of Array Declarations in C

1 Declaration Only (No Initialization)

📌 Values are **garbage**

```
int arr[5];    // garbage values
```

2 Compile-Time Initialization

📌 Values known **at compile time**

```
int arr[5] = {10, 20, 30, 40, 50};
```

✅ Most common

✅ Fast & safe

3 Partial Initialization

📌 Remaining elements automatically become **0**

```
int arr[5] = {1, 2};
```

🧠 Internally becomes:

```
{1, 2, 0, 0, 0}
```

4 Size Auto-Calculated (Size NOT Mentioned)

📌 Compiler calculates size automatically

```
int arr[] = {5, 10, 15, 20};
```

🧠 Size = 4 elements

5 All Zeros Initialization (IMPORTANT ★)

📌 All elements initialized to **0**

```
int arr[5] = {0};
```

🧠 Internally:

```
{0, 0, 0, 0, 0}
```

🔥 Very useful in contests & exams

6 Explicit All Zeroes (Less Preferred)

```
int arr[5] = {0, 0, 0, 0, 0};
```

⚠️ Correct but **not efficient**

7 Runtime Initialization (User Input)

📌 Values decided at **runtime**

```
int n;  
scanf("%d", &n);  
  
int arr[n];  
for(int i = 0; i < n; i++) {  
    scanf("%d", &arr[i]);  
}
```

⚠️ IMPORTANT RULES (Exam Gold 💎)

❌ This is **INVALID**

```
int n = 5;  
int arr[n] = {1,2,3,4,5}; // ❌ not compile-time constant
```

✅ Compile-time initialization **needs constant size**



Quick Comparison Table 🙌

Type	Size Known	Values
Declaration only	Yes	Garbage ❌
Compile-time init	Yes	Given values ✅
Partial init	Yes	Rest = 0 ✅
Auto size	No	Compiler decides ✅
All zeroes	Yes	All 0 ✅

Calculating Size of Array (IMPORTANT 🔥)

Total size (bytes)

```
sizeof(arr)
```

Number of elements

```
int n = sizeof(arr) / sizeof(arr[0]);
```

📌 Works only inside same scope (not in functions)

Traversing an Array

👉 Visiting each element using loop

```
for(int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
```

Taking Input from User

```
int n;
```

```
scanf("%d", &n);
```

```
int arr[n];  
for(int i = 0; i < n; i++) {  
    scanf("%d", &arr[i]);  
}
```

Printing Array Elements

```
for(int i = 0; i < n; i++) {  
    printf("%d ", arr[i]);  
}
```

Out of Bound Error (VERY IMPORTANT)

 Accessing index **outside valid range**

```
int arr[3] = {1, 2, 3};  
printf("%d", arr[3]); //  invalid
```

 Valid index range: 0 to size-1

 Causes **undefined behavior** / garbage value / crash

Common Mistakes

 Using wrong loop condition

```
for(int i = 0; i <= n; i++) // 
```

 Correct

```
for(int i = 0; i < n; i++) // 
```

✗ Assuming array size auto-known in function

```
void fun(int arr[]) {  
    sizeof(arr); // ✗ gives pointer size  
}
```

✗ Forgetting & in scanf

```
scanf("%d", arr[i]); // ✗  
scanf("%d", &arr[i]); // ✓
```

Static Size vs Logical Size of Array (VERY IMPORTANT)



Static Size of Array

- 📌 Static size = Total memory allocated for the array
- 📌 Fixed at **compile time**
- 📌 **Cannot change**

```
int arr[10]; // static size = 10
```

🧠 Means:

➡ Memory reserved for **10 integers**, even if you don't use all

Logical Size of Array

- 📌 Logical size = Number of elements actually used
- 📌 Decided at **runtime**
- 📌 Can be $\leq$ **static size**

```
int arr[10];  
int n = 4; // logical size = 4
```

```
arr[0] = 10;  
arr[1] = 20;  
arr[2] = 30;  
arr[3] = 40;
```

🧠 Only first 4 elements are meaningful

vs Static Size vs Logical Size (Difference Table)

Feature	Static Size 🧱	Logical Size 🧠
Meaning	Total capacity	Actual elements used
When decided	Compile time	Runtime
Changeable	❌ No	✅ Yes
Used in loops	❌ Never	✅ Always
Memory impact	Fixed	No new memory

🔄 Correct Traversal (USING LOGICAL SIZE ✅)

```
for(int i = 0; i < n; i++) {  
    printf("%d ", arr[i]);  
}
```

❌ WRONG Traversal (USING STATIC SIZE ❌)

```
for(int i = 0; i < 10; i++) {    // ❌ may print garbage  
    printf("%d ", arr[i]);  
}
```

⚠ Common Mistakes

- ✗ Confusing static size with logical size
 - ✗ Looping till static size
 - ✗ Printing garbage values
-

🔧 Example: Inserting Elements Safely

```
int arr[10];  
int n = 0;    // logical size
```

```
arr[n++] = 5;  
arr[n++] = 10;  
arr[n++] = 15;
```

🧠 Logical size increases safely

? Why Arrays Use 0-Based Indexing?

🔑 Core Reason (MOST IMPORTANT 🔥)

👉 Array indexing is based on memory address calculation, not counting.

💡 Formula used internally:

Address of `arr[i]` = `BaseAddress + (i × size_of_datatype)`

✓ When `i = 0`

➡ Address = `BaseAddress + 0`

➡ Points to the first element directly

🧠 Example (Memory Level Understanding)

```
int arr[5] = {10, 20, 30, 40, 50};
```

Assume:

- Base address = 1000
- Size of int = 4 bytes

Index	Address Calculation	Actual Address
0	$1000 + (0 \times 4)$	1000
1	$1000 + (1 \times 4)$	1004
2	$1000 + (2 \times 4)$	1008

 **Index = offset from base address**

Meaning of Index in C

 Index **does NOT** mean position number

 Index means **how many memory blocks away from base**

```
arr[i]  ≡  *(arr + i)
```

✓ $arr[0] = *(arr + 0) \rightarrow$ first element

✓ $arr[1] = *(arr + 1) \rightarrow$ next element

Why NOT 1-Based Indexing?

If indexing started from 1:

```
Address = BaseAddress + (1 × size)
```

- ✗ First element would be **wasted or skipped**
 - ✗ Extra subtraction needed internally
 - ✗ Slower & complex hardware logic
-

Performance & Hardware Reason

✓ 0-based indexing:

- No extra computation
- Faster pointer arithmetic
- Direct mapping with memory

🔥 That's why **C, C++, Java, Python** use 0-based indexing

Mathematical Advantage

Many algorithms become simpler:

- ✓ Length = n
- ✓ Last index = $n-1$
- ✓ Loop condition:

```
for(int i = 0; i < n; i++)
```

✗ With 1-based indexing → more confusion & off-by-one errors

Historical Reason

- 📖 C was built on **low-level memory + pointers**
- 📖 Designed to stay close to **hardware**

➡ 0-based indexing fits perfectly with pointer arithmetic

Key Properties of Arrays

- ✨ Fixed size
 - ✨ Same data type
 - ✨ Contiguous memory
 - ✨ Fast access ($O(1)$)
 - ✨ Cannot grow/shrink dynamically
-

Example: Sum of Array

```
int sum = 0;
for(int i = 0; i < n; i++) {
    sum += arr[i];
}
printf("%d", sum);
```